

6.5830/1 Lecture 20: Cloud DB

4/28/2026

Tianyu Li

litianyu@mit.edu

Based on materials from Xiangyao Yu,
Sam Madden, and others

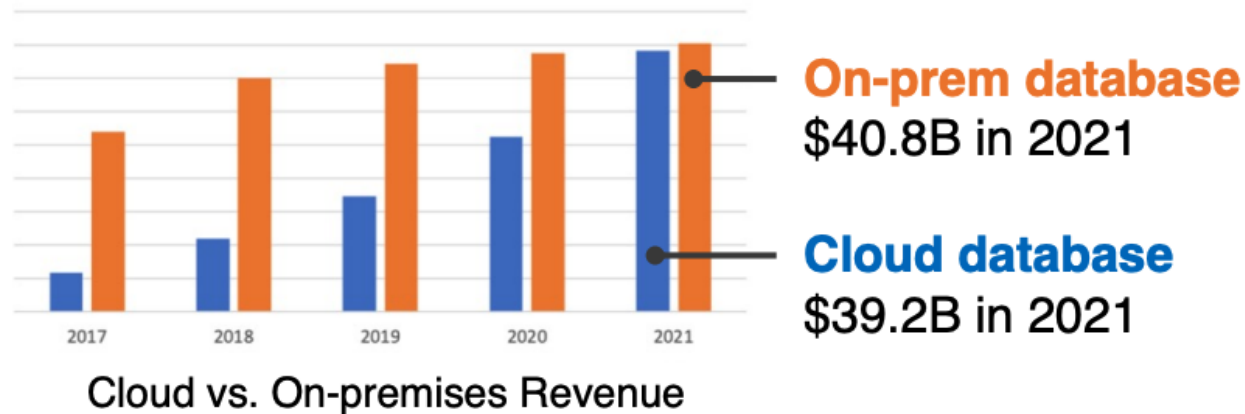
Administrivia

- PSet 3 due today
- Quiz 2 will be released next week
 - Some make-ups are in progress
 - Email staff now if you still need to make up quiz 2 or lab 3!
- Reminder: Final project due May 7, Lab 4 + writeup due May 12
 - No late days!

Cloud DB

According to Gartner Report ^[1]

\$39.2 billion, 49% of all DBMS revenue from cloud in 2021



Low Cost

Elasticity

Availability

[1] DBMS Market Transformation 2021: The Big Picture, <https://blogs.gartner.com/merv-adrian/2022/04/16/dbms-market-transformation-2021-the-big-picture/>

Cloud DB

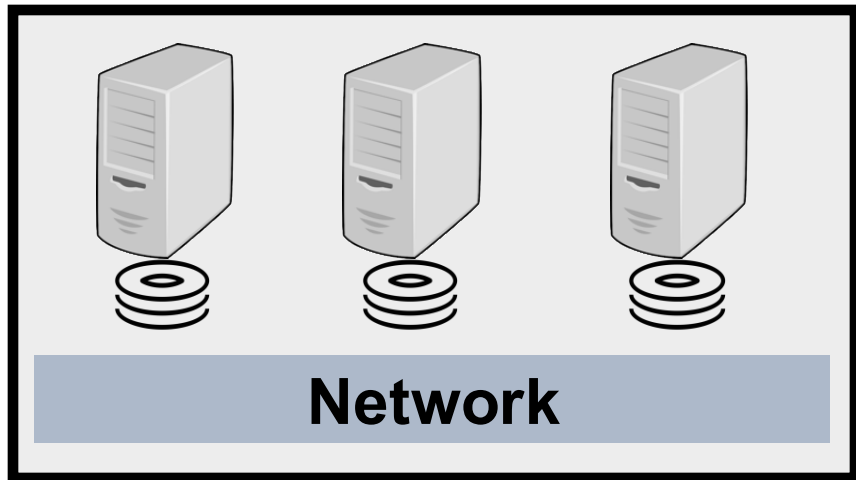
- Traditional Deployment Model: on-premise databases
 - Host and maintain your own hardware
 - Purchase software licenses / deploy open-source
 - Hire DBAs to deploy and manage daily workloads
- Problem: a lot of technical work
 - Ideally, customers want a lot of this complexity managed

First: Easy Solutions

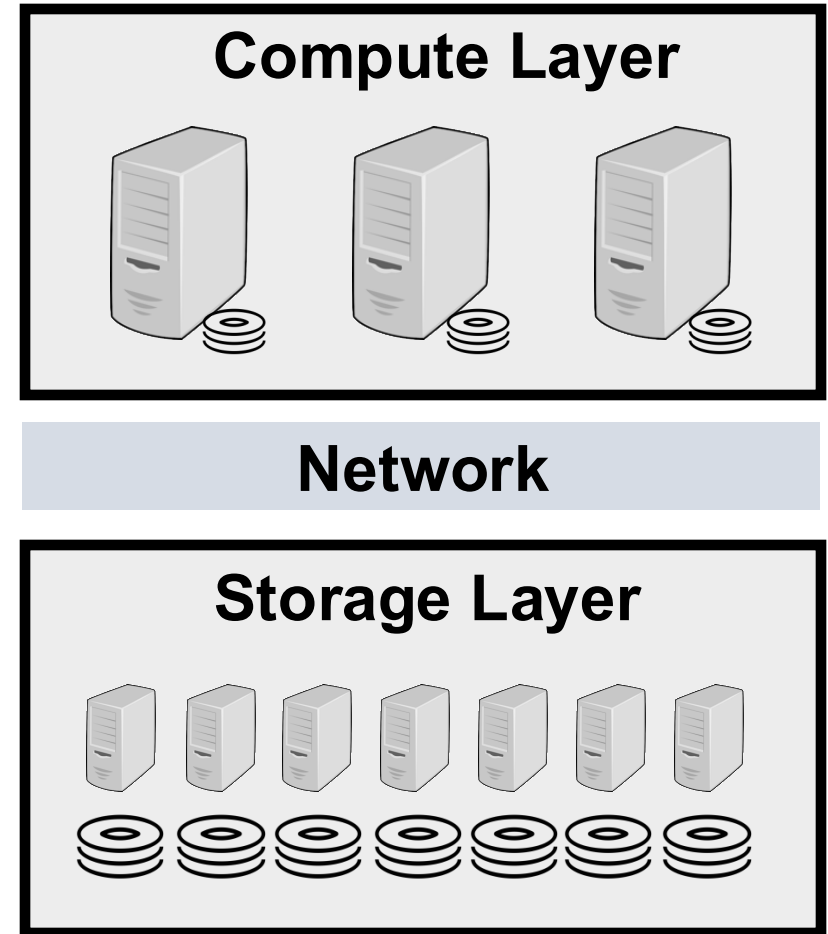
- Take DB software, and instead of deploying on-prem, deploy them onto VMs
 - Reasonable solution that is often used
- Problems:
 - Software complexity still exists
 - Performance and fault-tolerance challenges
 - Misses on many opportunities provided by the cloud (elasticity, high availability, economy of scale, etc.)

What's New with DBaaS?

Shared Nothing

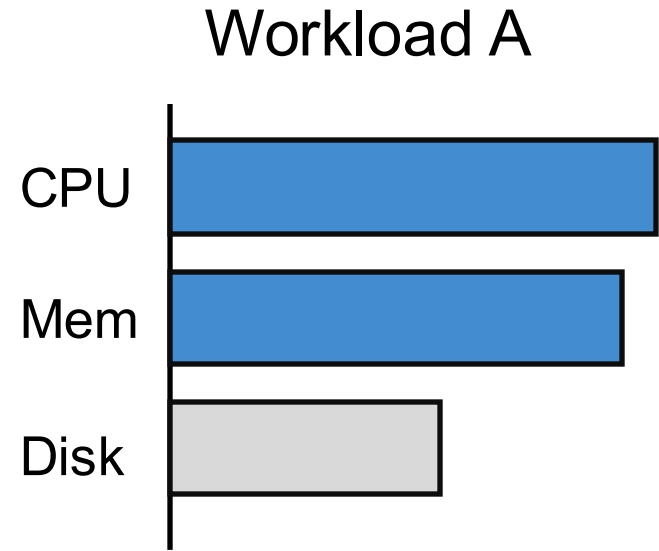
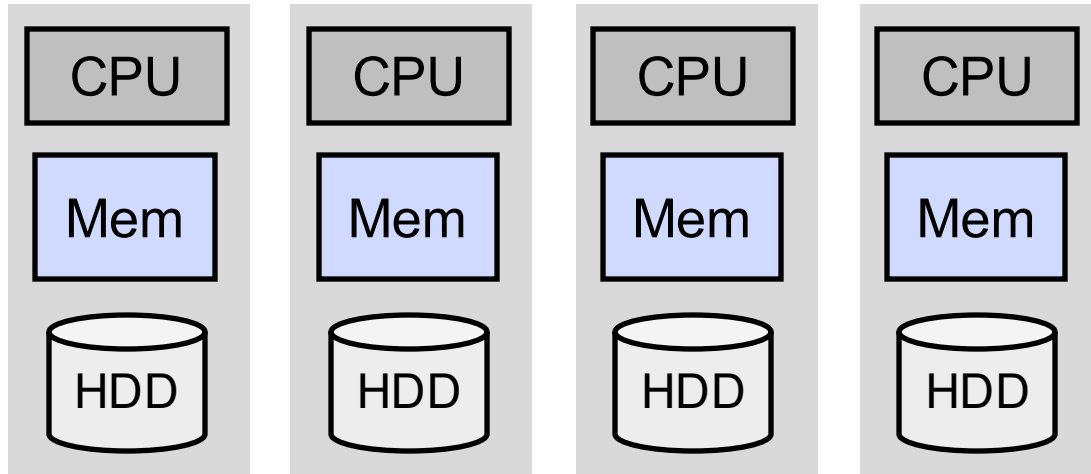


Storage disaggregation

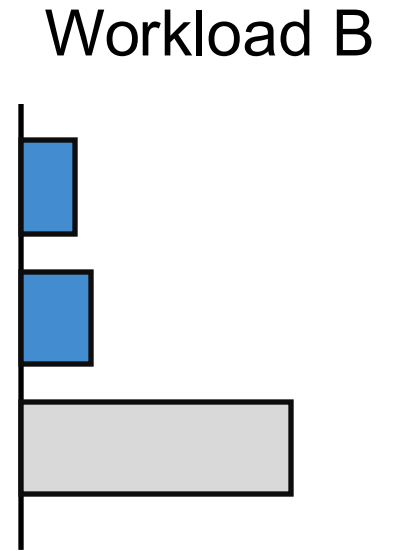


Disaggregation $\approx$ **A cluster of specialized shared-nothing clusters**

Shared Nothing – Disadvantages



More CPU intensive

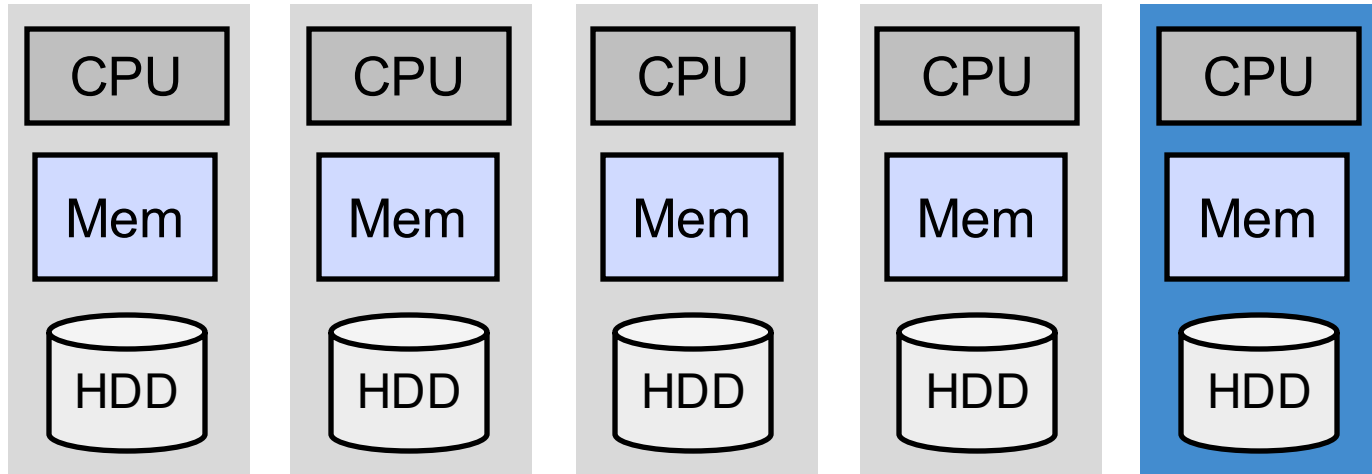


Less CPU intensive

Heterogeneous workload

- Static resource provisioning cannot adjust to heterogeneous workloads
- Must pay for entire cluster even when no queries exist

Shared Nothing – Disadvantages

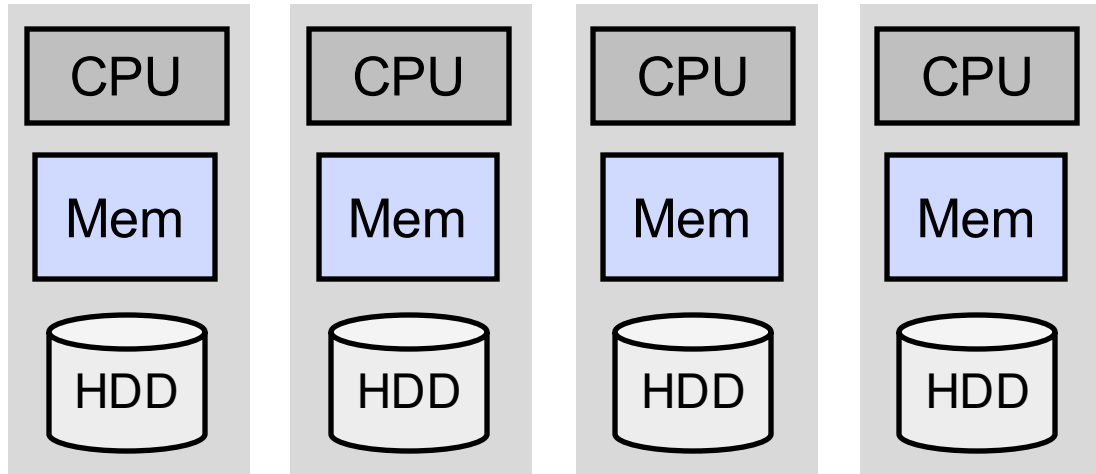


Heterogeneous workload

Membership changes

- Add/Remove a node: data redistribution
- Significant hassle that may cause down time

Shared Nothing – Disadvantages



Heterogeneous workload

Membership changes

Online upgrade

- Similar to membership change but affect all nodes

Storage Disaggregation Architecture

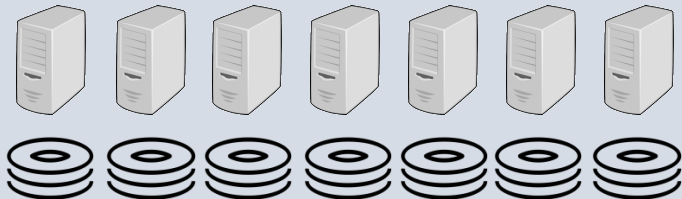
Why disaggregate? Compute and storage behave differently

Compute Layer



Network

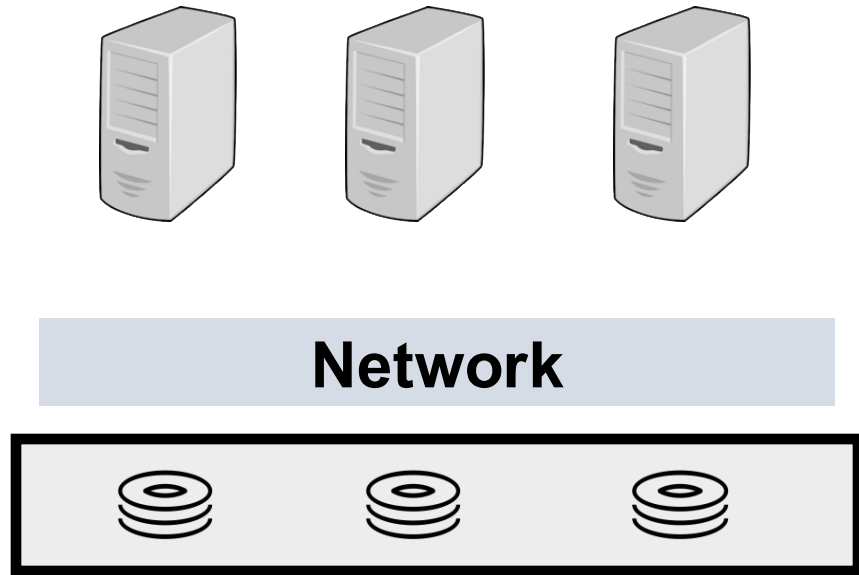
Storage Layer



- **Stateless** → easy to scale
 - **Expensive** → VMs cost **\$0.1–10/hour**
 - **Bursty** → demand changes rapidly
-
- **Stateful** → difficult to scale
 - **Low-cost** → S3 costs **\$0.02/GB/month**
 - **Steady** → demand changes slowly

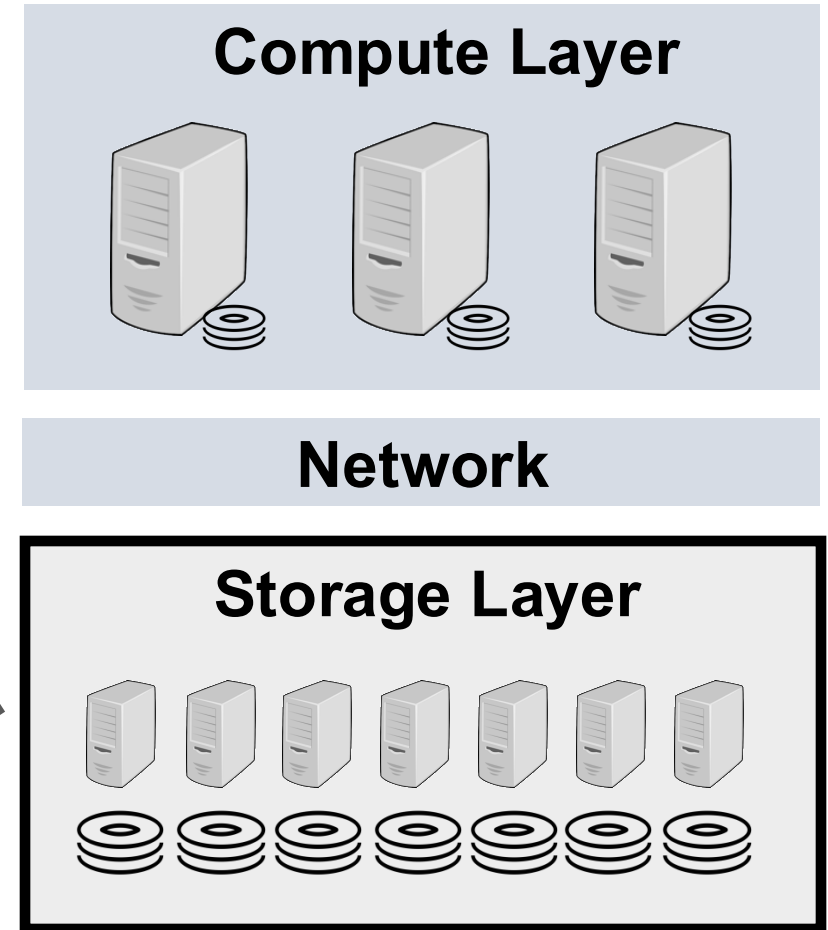
Storage Disaggregation vs. Shared Disk?

Shared Disk

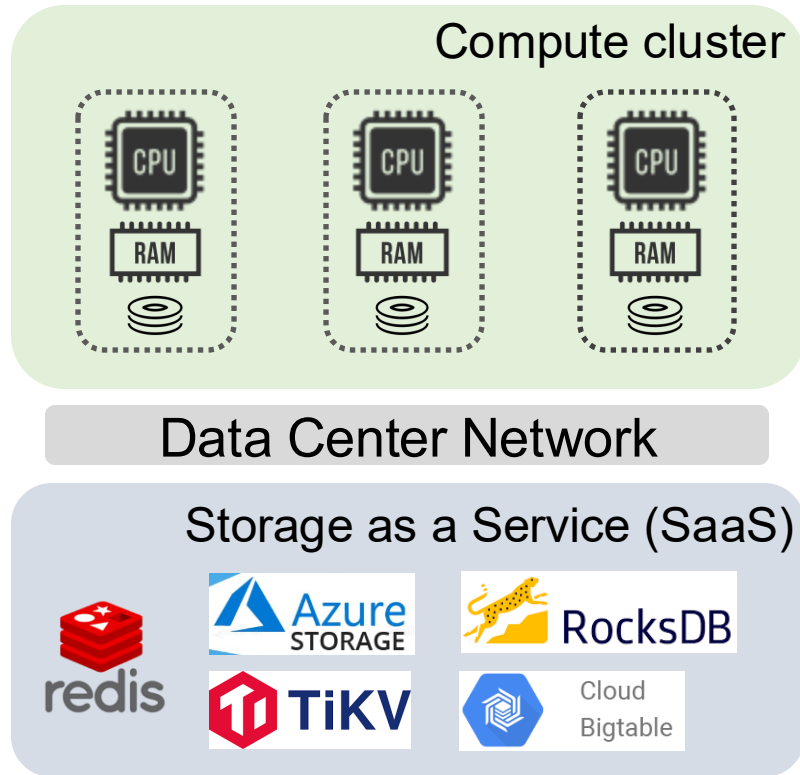


Storage service offers: **data replication**, **horizontal scaling**, **advanced APIs**, etc.

Storage disaggregation

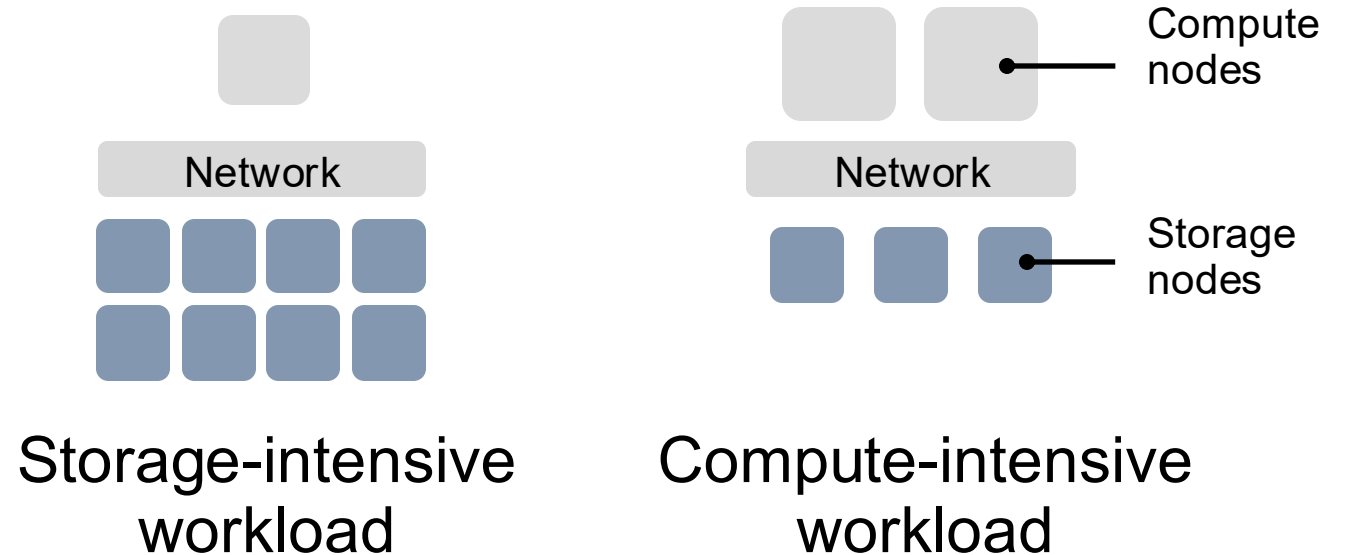


Advantages of Storage-Disaggregation

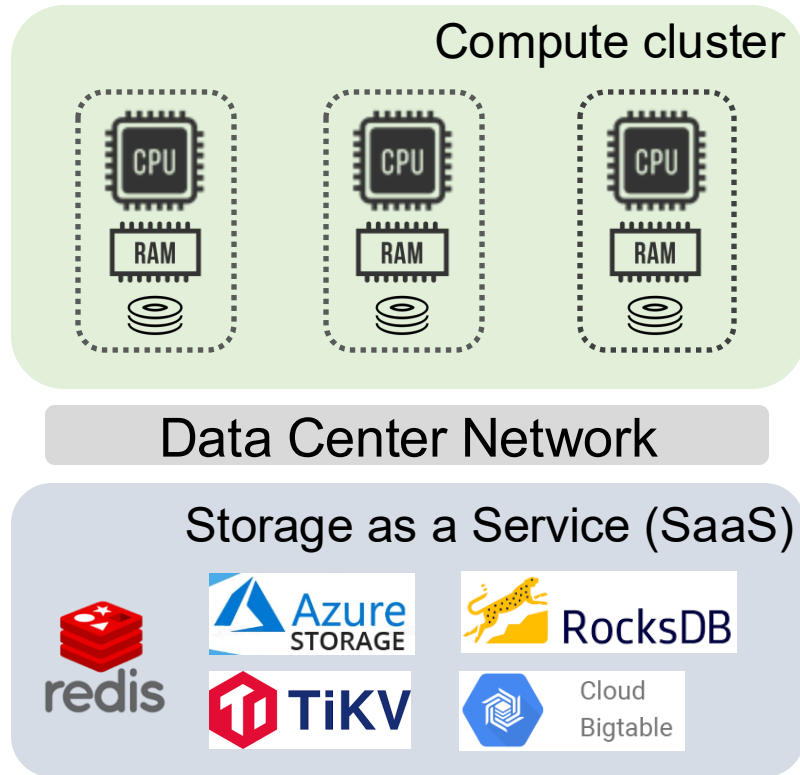


Advantage #1: Independent Scaling

- Compute and storage resources can scale independently



Advantages of Storage-Disaggregation



Advantage #2: Elastic Scaling of Compute

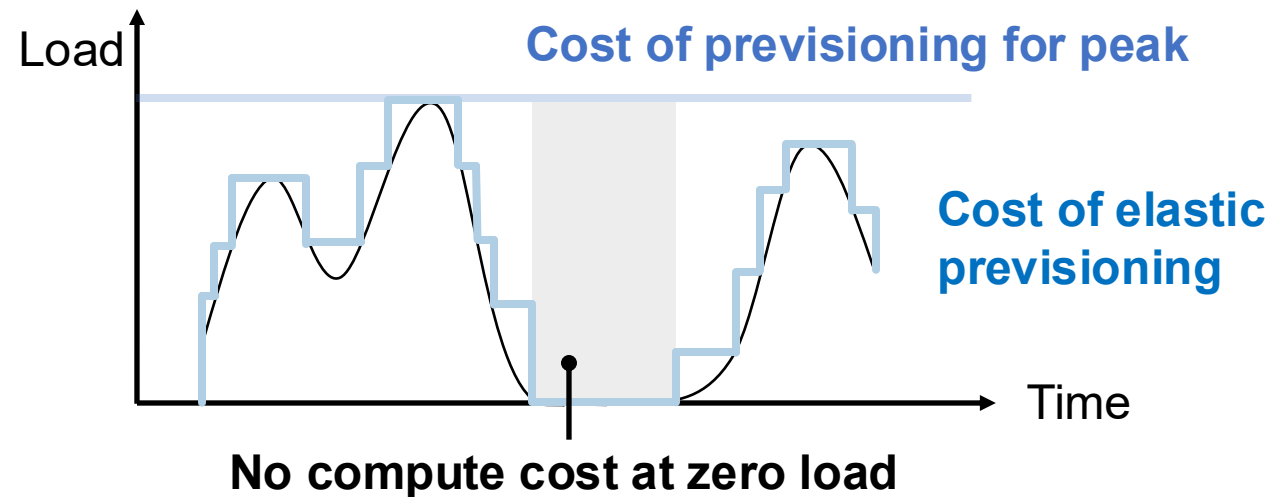
- Storage service can be much cheaper than compute servers

S3 storage price

\$0.02 per GB per month

16 vCPU Virtual Machine

\$0.5 per hour per VM



Today

- Cloud-native OLAP (Data Warehouse)
 - Case Study: Snowflake
- Cloud-native OLTP
 - Case Study: Amazon Aurora

Snowflake

The Snowflake Elastic Data Warehouse

Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes,
Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel,
Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley,
Peter Povinec, Greg Rahn, Spyridon Triantafyllis, Philipp Unterbrunner

Snowflake Computing

ABSTRACT

We live in the golden age of distributed computing. Public cloud platforms now offer virtually unlimited compute and storage resources on demand. At the same time, the Software-as-a-Service (SaaS) model brings enterprise-class systems to users who previously could not afford such systems due to their cost and complexity. Alas, traditional data warehousing systems are struggling to fit into this new environment. For one thing, they have been designed for fixed resources and are thus unable to leverage the cloud's elasticity. For another thing, their dependence on complex ETL pipelines and physical tuning is at odds with the flexibility and freshness requirements of the cloud's new types of semi-structured data and rapidly evolving workloads.

We decided a fundamental redesign was in order. Our mission was to build an enterprise-ready data warehousing solution for the cloud. The result is the Snowflake Elastic Data Warehouse, or "Snowflake" for short. Snowflake is a multi-tenant, transactional, secure, highly scalable and elastic system with full SQL support and built-in extensions for semi-structured and schema-less data. The system is offered as a pay-as-you-go service in the Amazon cloud. Users upload their data to the cloud and can immediately manage and query it using familiar tools and interfaces. Implementation began in late 2012 and Snowflake has been generally available since June 2015. Today, Snowflake is used in production by a growing number of small and large organizations alike. The system runs several million queries per day over multiple petabytes of data.

In this paper, we describe the design of Snowflake and its novel multi-cluster, shared-data architecture. The paper highlights some of the key features of Snowflake: extreme elasticity and availability, semi-structured and schema-less data, time travel, and end-to-end security. It concludes with lessons learned and an outlook on ongoing work.

Categories and Subject Descriptors

Information systems [Data management systems]: Database management system engines

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGMOD/PODS'16 June 26 - July 01, 2016, San Francisco, CA, USA

© 2016 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-3531-7/16/06.

DOI: <http://dx.doi.org/10.1145/2882903.2903741>

Keywords

Data warehousing, database as a service, multi-cluster shared data architecture

1. INTRODUCTION

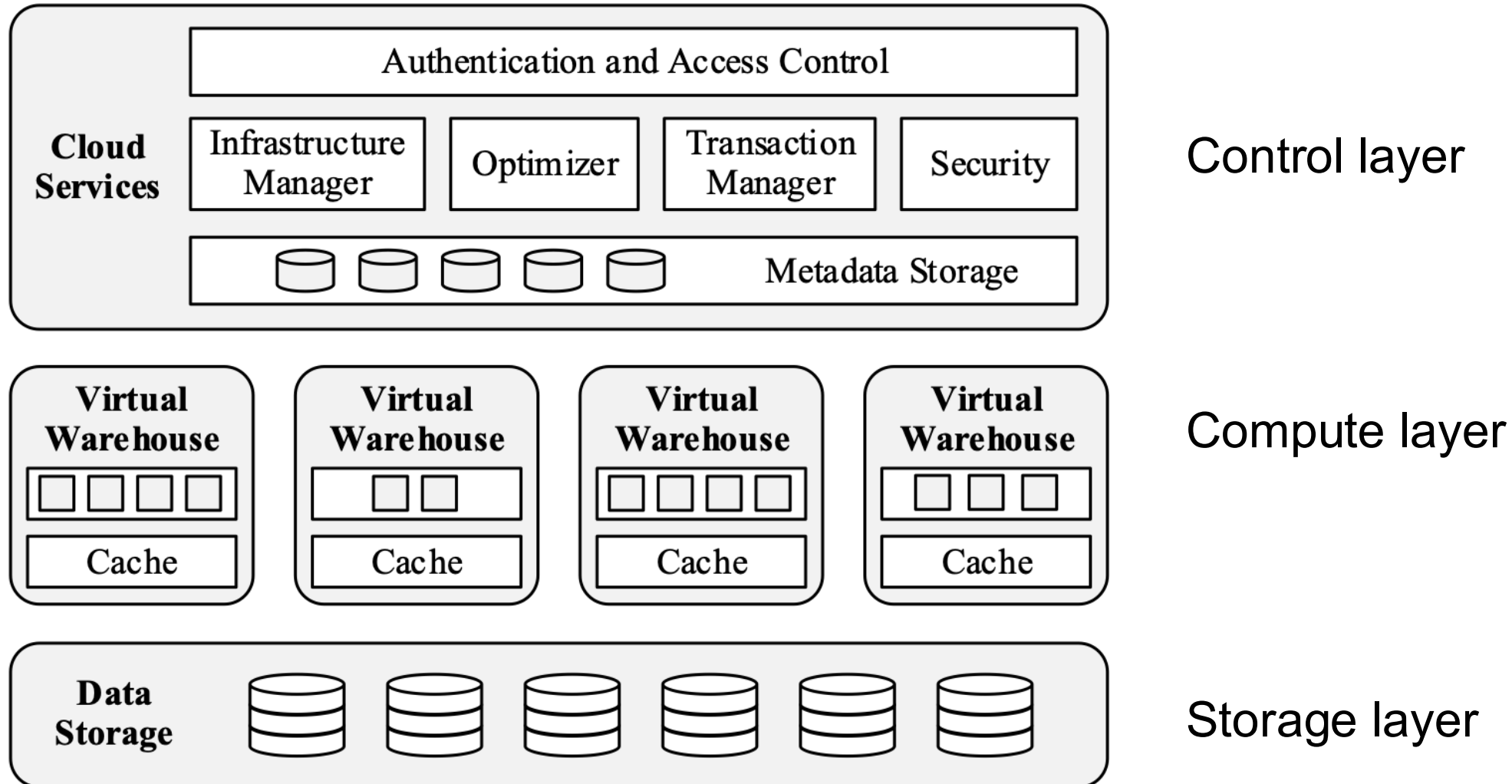
The advent of the cloud marks a move away from software delivery and execution on local servers, and toward shared data centers and software-as-a-service solutions hosted by platform providers such as Amazon, Google, or Microsoft. The shared infrastructure of the cloud promises increased economies of scale, extreme scalability and availability, and a pay-as-you-go cost model that adapts to unpredictable usage demands. But these advantages can only be captured if the *software* itself is able to scale elastically over the pool of commodity resources that is the cloud. Traditional data warehousing solutions pre-date the cloud. They were designed to run on small, static clusters of well-behaved machines, making them a poor architectural fit.

But not only the platform has changed. Data has changed as well. It used to be the case that most of the data in a data warehouse came from sources within the organization: transactional systems, enterprise resource planning (ERP) applications, customer relationship management (CRM) applications, and the like. The structure, volume, and rate of the data were all fairly predictable and well known. But with the cloud, a significant and rapidly growing share of data comes from less controllable or external sources: application logs, web applications, mobile devices, social media, sensor data (Internet of Things). In addition to the growing volume, this data frequently arrives in schema-less, semi-structured formats [8]. Traditional data warehousing solutions are struggling with this new data. These solutions depend on deep ETL pipelines and physical tuning that fundamentally assume predictable, slow-moving, and easily categorized data from largely internal sources.

In response to these shortcomings, parts of the data warehousing community have turned to "Big Data" platforms such as Hadoop or Spark [8, 11]. While these are indispensable tools for data center-scale processing tasks, and the open source community continues to make big improvements such as the Stinger Initiative [48], they still lack much of the efficiency and feature set of established data warehousing technology. But most importantly, they require significant engineering effort to roll out and use [16].

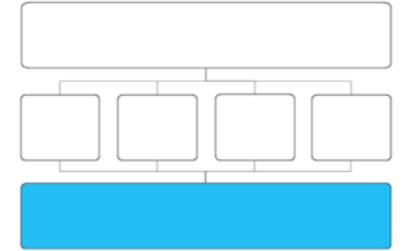
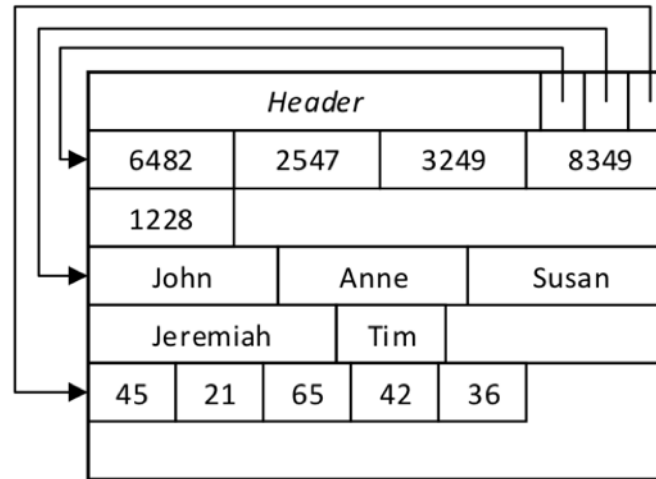
We believe that there is a large class of use cases and workloads which can benefit from the economics, elasticity, and service aspects of the cloud, but which are not well served by either traditional data warehousing technology or

Multi-Cluster Shared-Data Architecture



Architecture – Storage

Data format: PAX

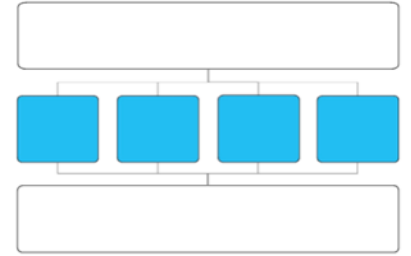


Data horizontally partitioned into immutable files (~16MB)

- An update = remove and add an entire file
- Queries download file headers and columns they are interested in

For our purposes, this is just a standard storage service like S3

Architecture – Virtual Warehouse



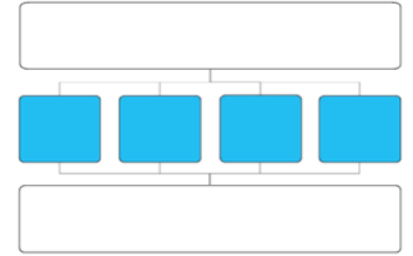
Elasticity and Isolation

- Created, destroyed, or resized at any point (may shutdown all VWs)
- User may create multiple VWs for multiple queries
- Determine the VW size based on performance and cost requirements

Architecture – Virtual Warehouse

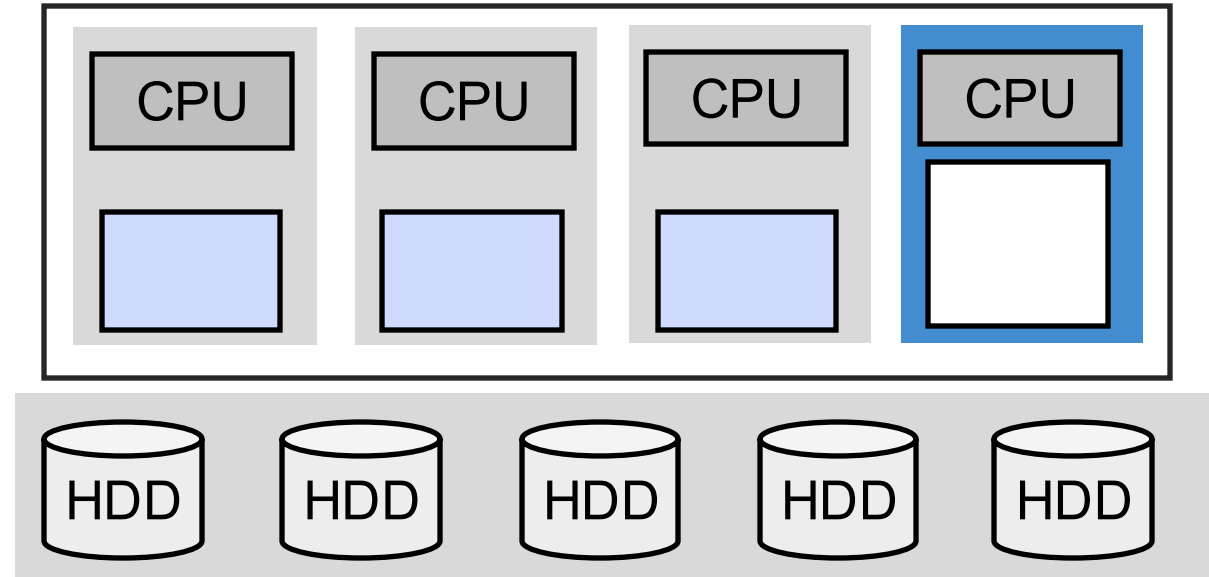
Local caching

- S3 data can be cached in local memory or disk



Consistent hashing

- When the hash table (n keys and m slots) is resized, only n/m keys need to be remapped
- When a VW is resized, **no data shuffle required**; rely on LRU to replace cache content



Architecture – Cloud Services

Multi-tenant layer shared across multiple users

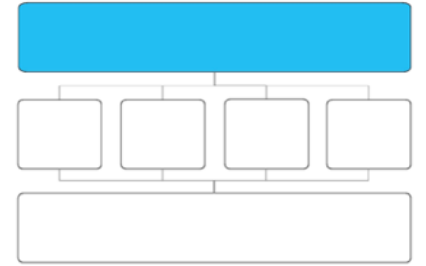
Query optimization

Concurrency control

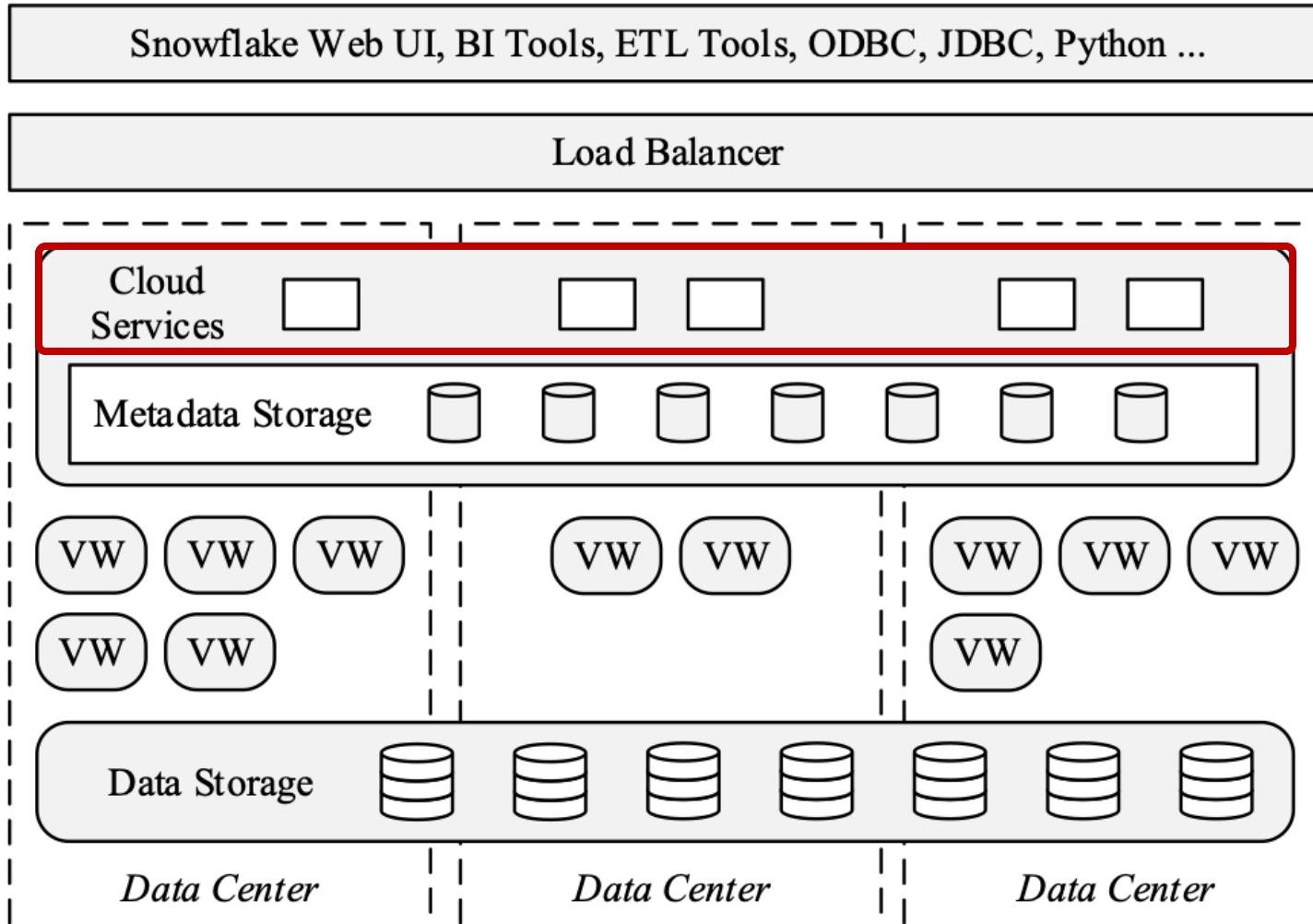
- Isolation: snapshot isolation (SI)
- S3 data is immutable, update entire files with MVCC
- Versioned snapshots used for time traveling
- Assumption: bulk data loading rather than OLTP

Pruning

- Snowflake has no index (same as some other data warehousing systems)
- Min-max based pruning: store min and max values for a data block

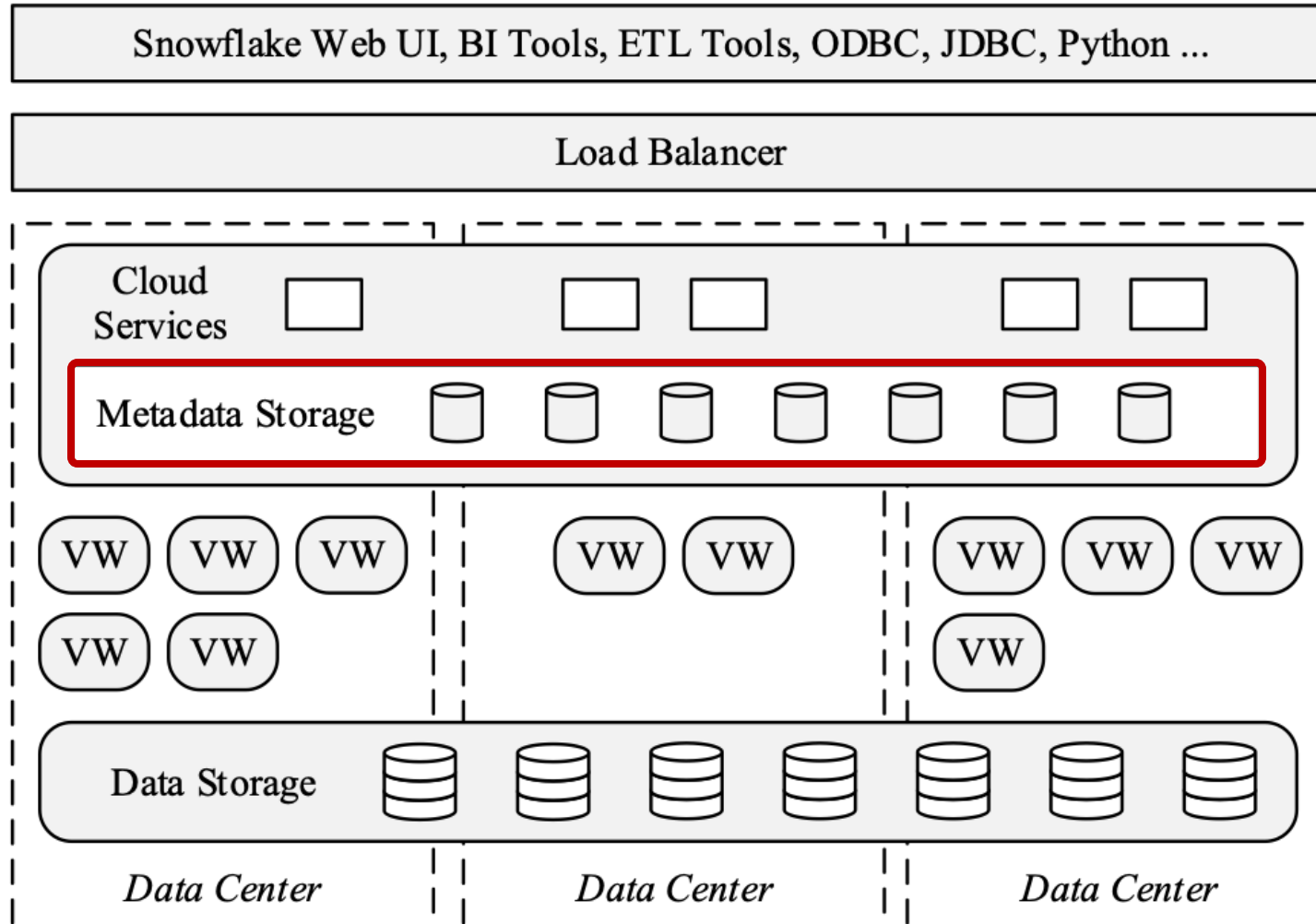


High Availability and Fault Tolerance



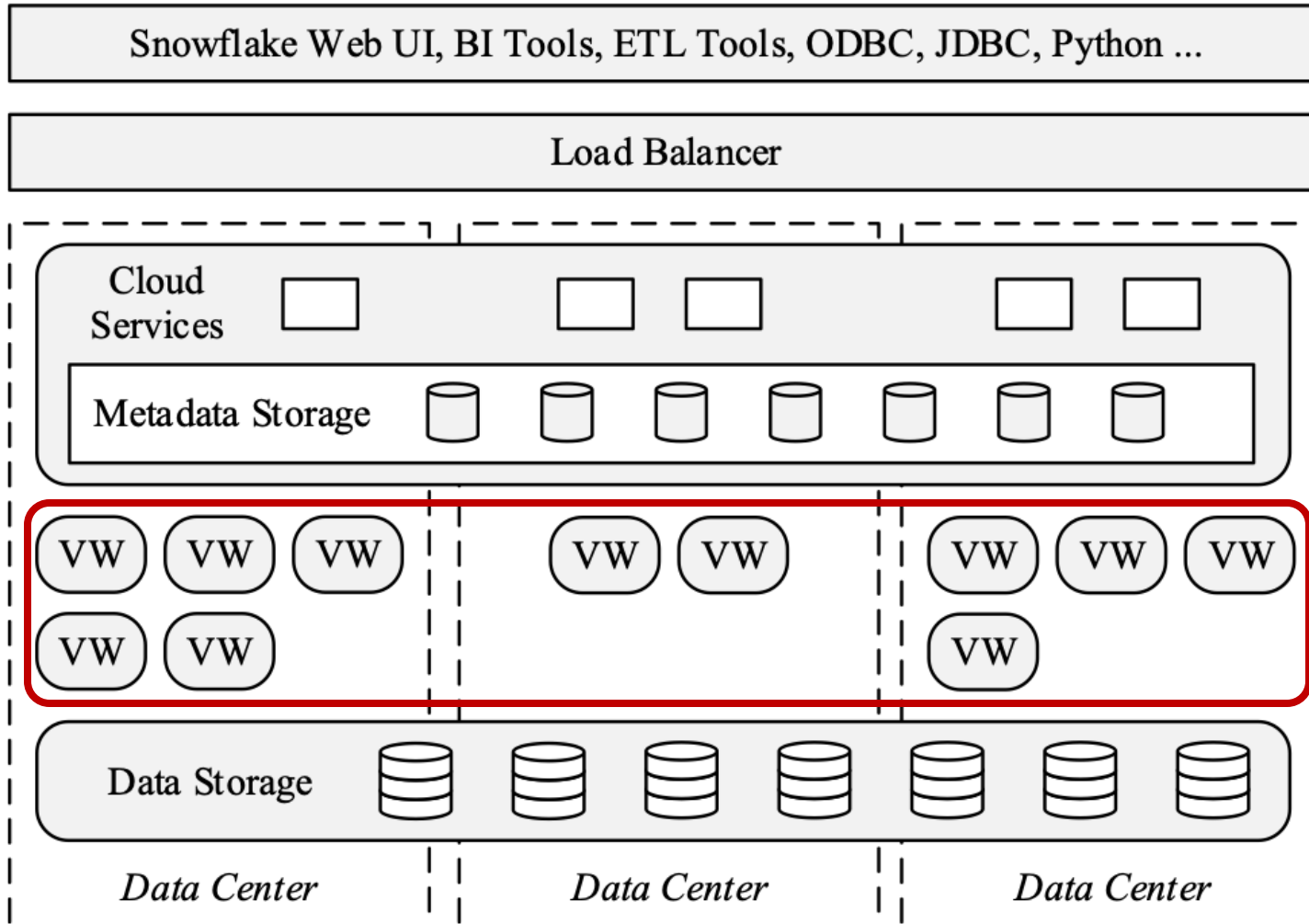
Stateless services

High Availability and Fault Tolerance



Replicated metadata
(FoundationDB)

High Availability and Fault Tolerance



One node failure in VW

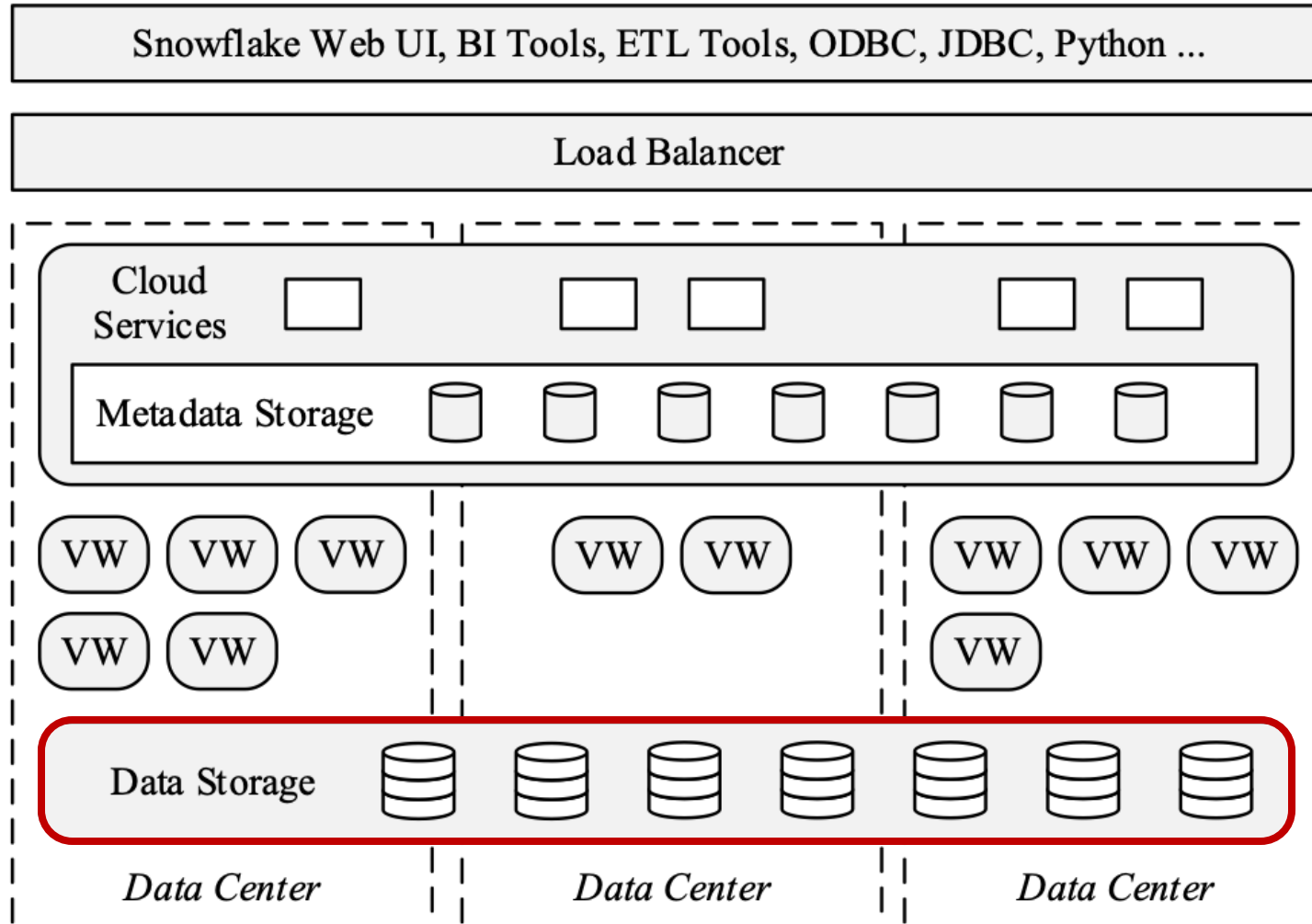
- Re-execute with failed node immediately replaced
- Re-execute with reduced number of nodes

Whole AZ failure

- Re-execute by re-provisioning a new VW

Hot-standby nodes

High Availability and Fault Tolerance



S3 is highly available and durable

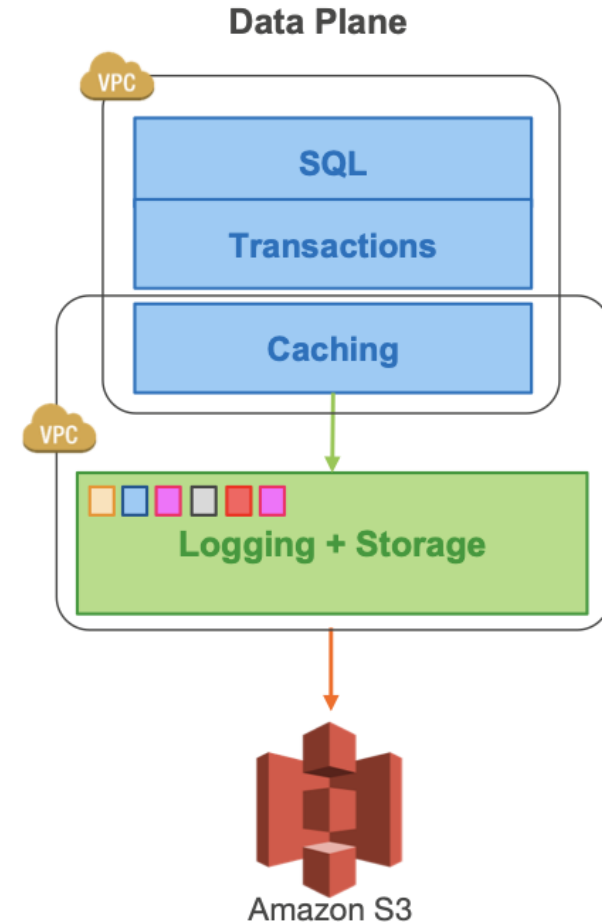
Next: Cloud OLTP

- Cloud storage (e.g. S3) does not behave like normal storage disks
 - On the one hand, it has massive aggregate throughput
 - On the other hand, it is much higher latency and potentially rate limited for single user
- For OLTP workload – must mask latency of naïve migration

Computation Pushdown in Cloud OLTP



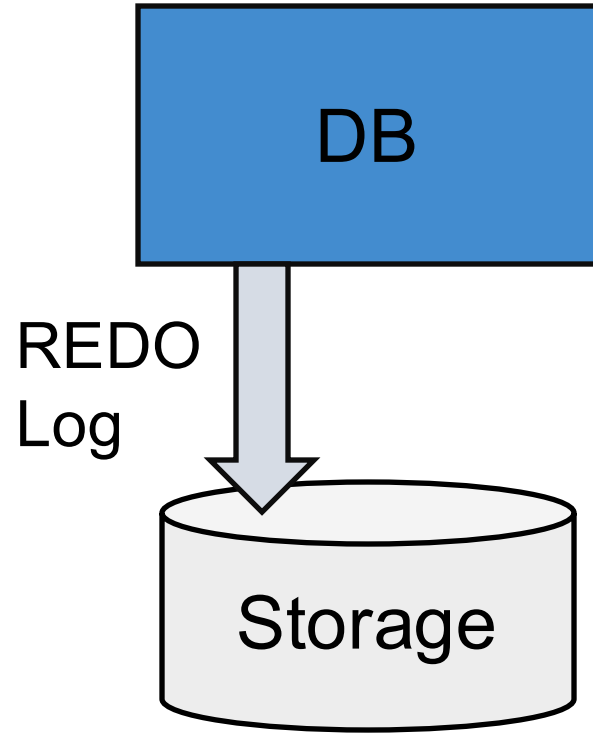
Idea: Push redo processing into the storage service



Amazon Aurora

- Base System: MySQL or Postgres
- Base DB writes **both log and data pages** to storage
- Aurora: modifies base DB to write **only REDO log** to storage
 - Buffer Pool reads from storage service
 - Buffer Pool discards pages on flush
 - WAL are replicated in the storage service
 - The storage layer replays the log into data pages

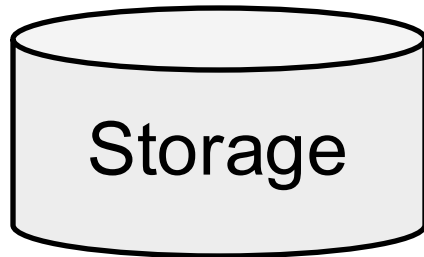
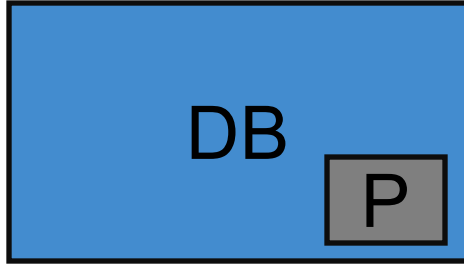
Forward Processing – Write and Commit



Write: flush REDO log record to storage (no UNDO)

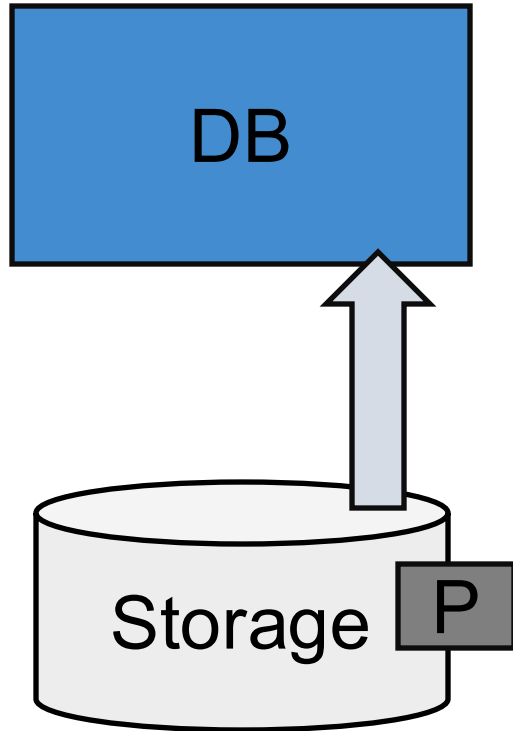
Commit: flush COMMIT log to storage

Forward Processing – Read



Buffer hit: read from main memory of the DB server

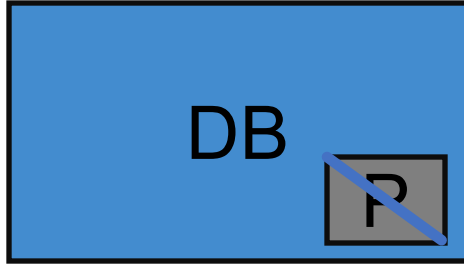
Forward Processing – Read



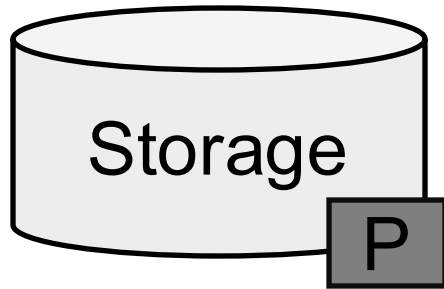
Buffer hit: read from main memory of the DB server

Buffer miss: read page from storage

Forward Processing – Eviction



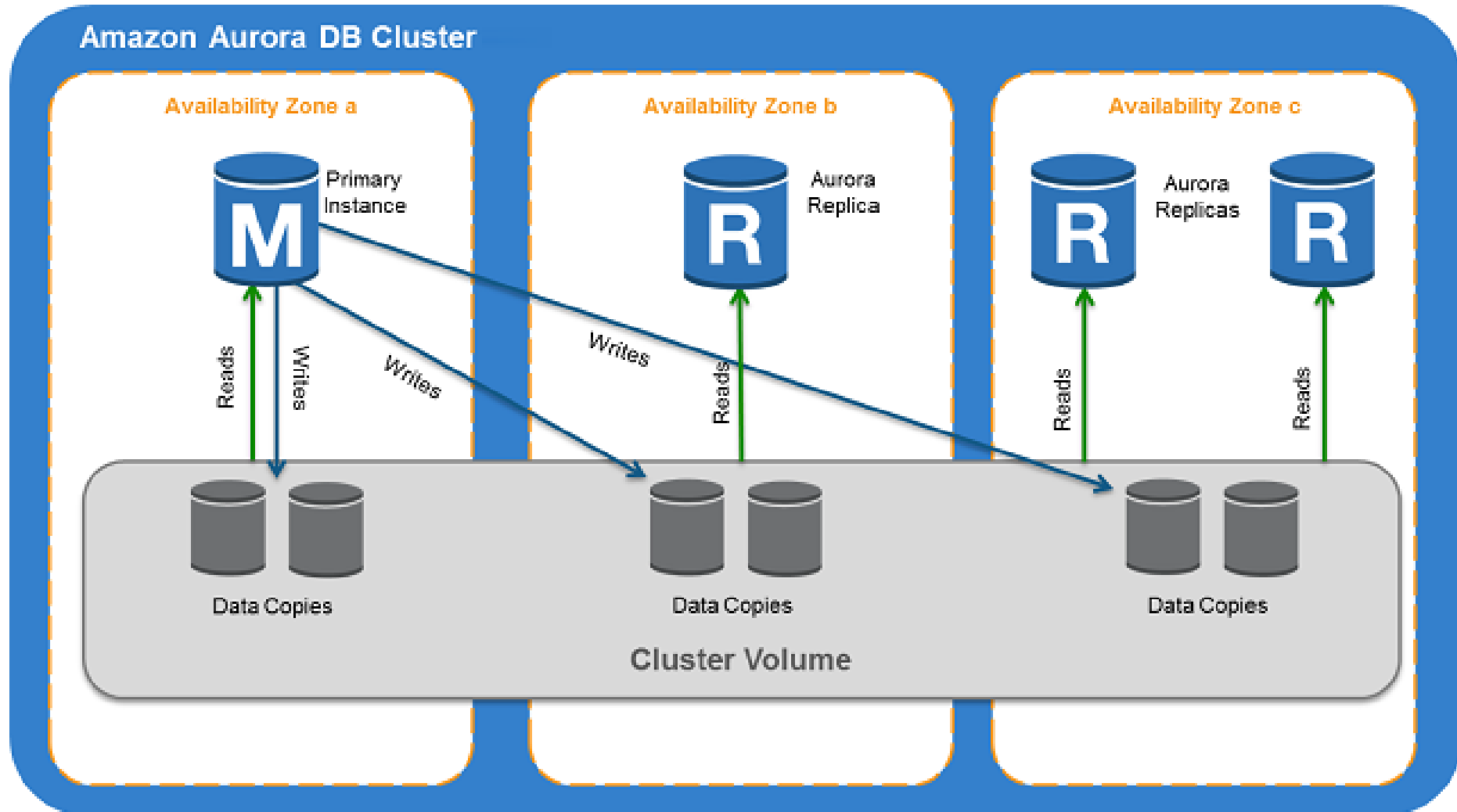
Aurora:
discard dirty
page



Dirty eviction: discard dirty page (no write back to storage)

- The page in storage will be updated through replaying the REDO log

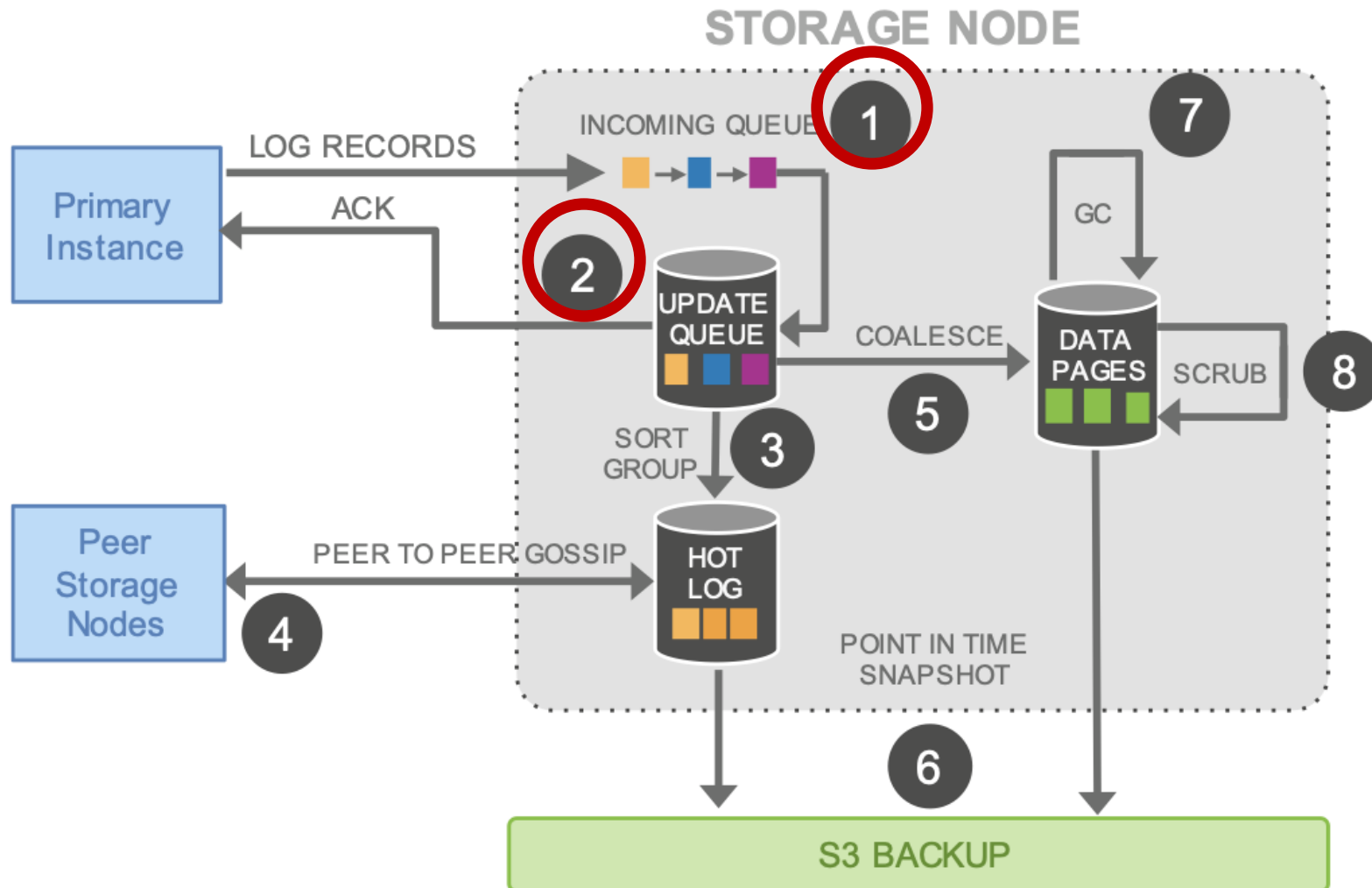
Aurora – Single Master



Aurora – Single Master

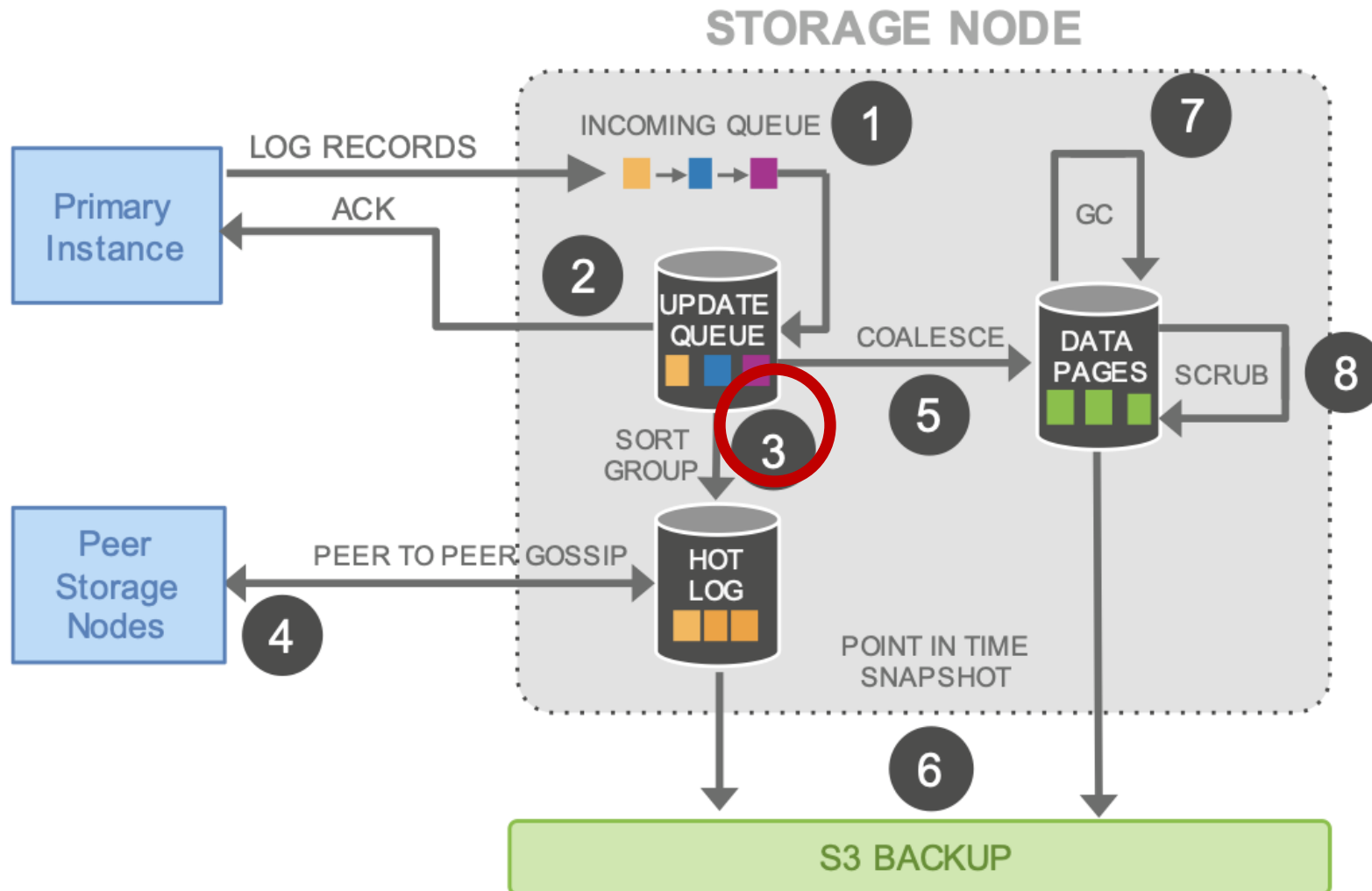
- Quorum write
 - Strongly consistent, but resilient to latency spikes caused by stragglers
- When is quorum read necessary?
 - Only in recovery
- Storage nodes gossip to keep in sync
 - Easier than in Dynamo case because primary decides on log ordering

Storage Node



Only Steps 1 & 2 are in the foreground path

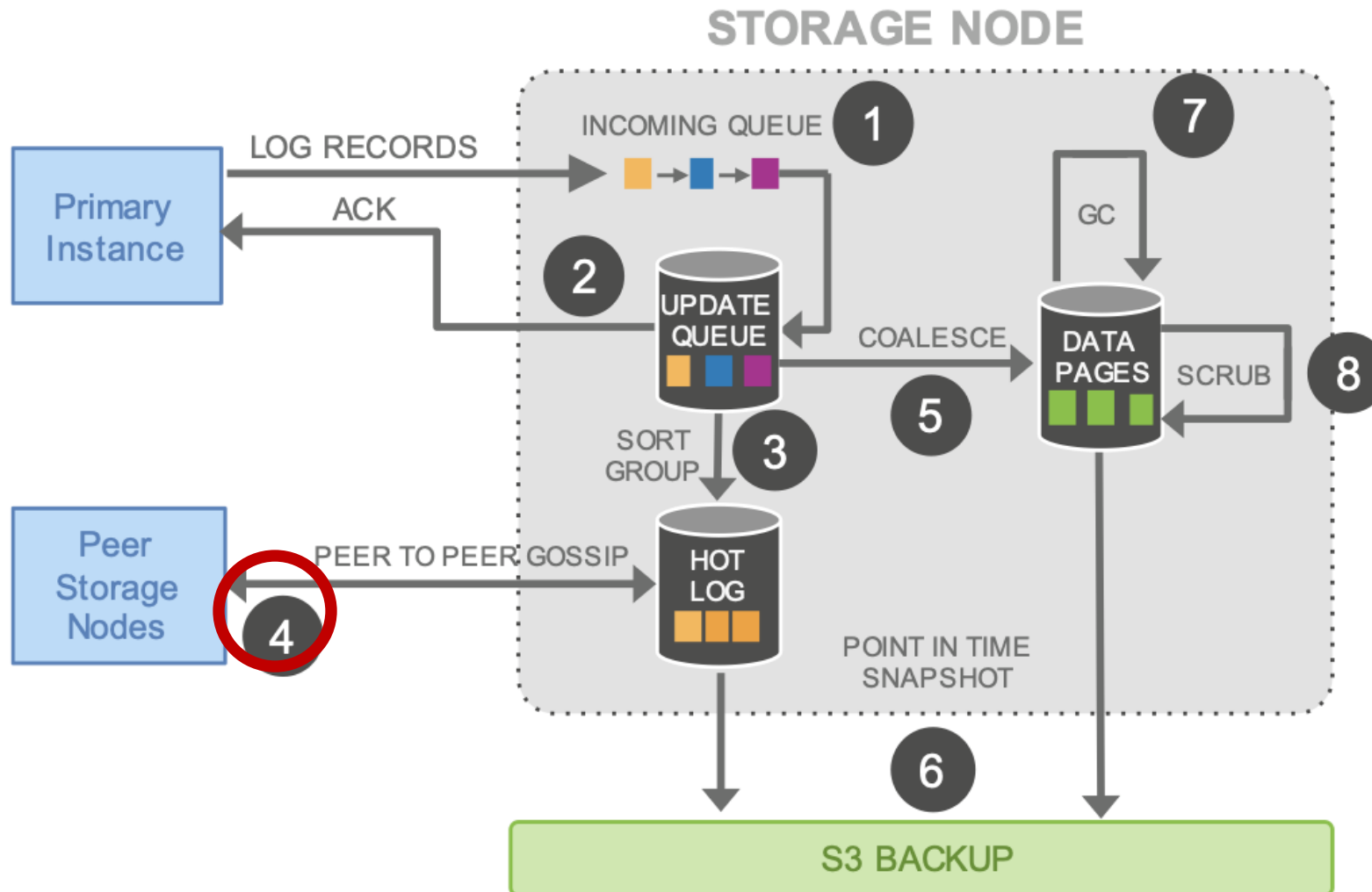
Storage Node



Identify gaps in the log



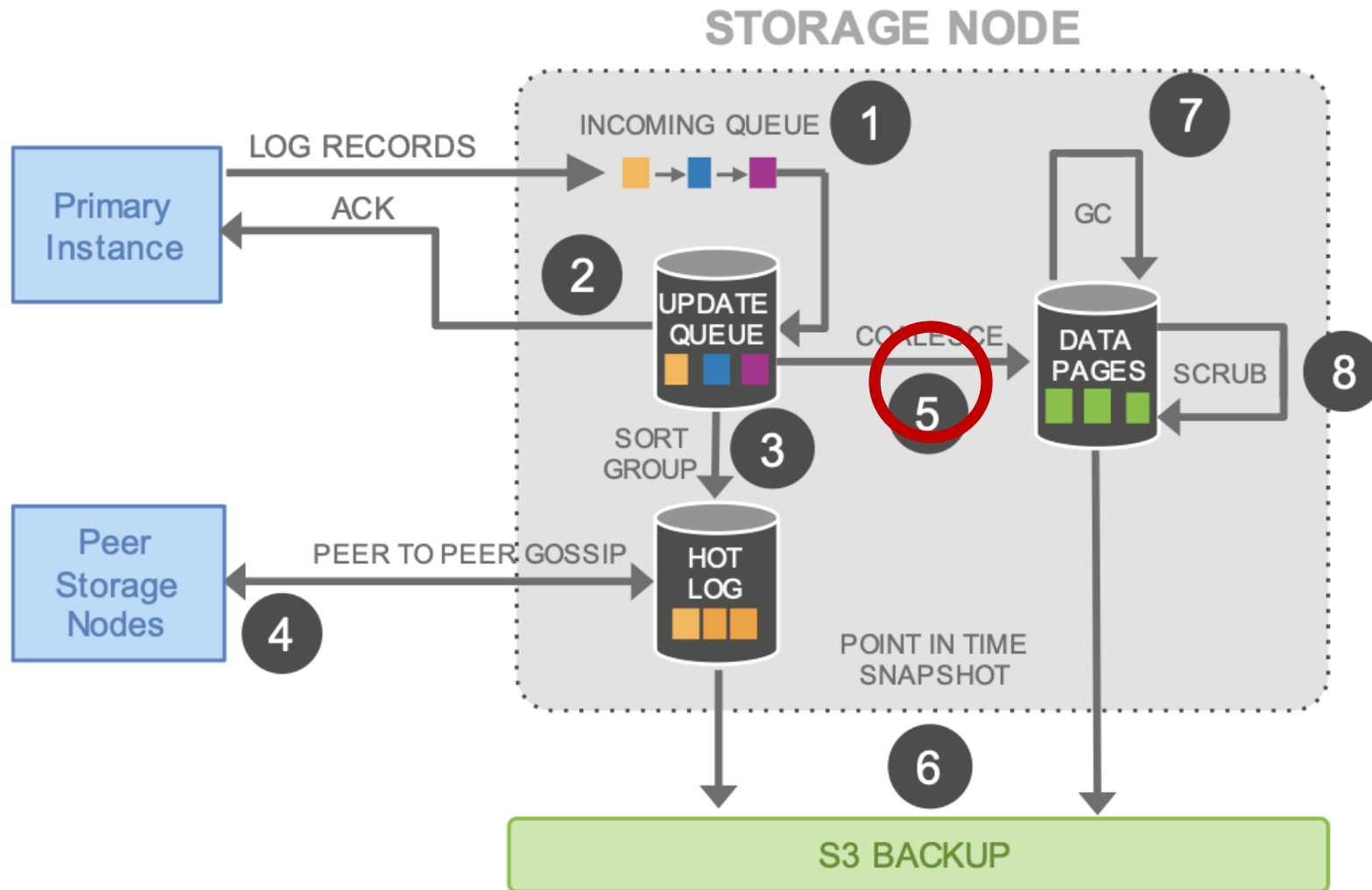
Storage Node



Gossip with peers to fill gaps

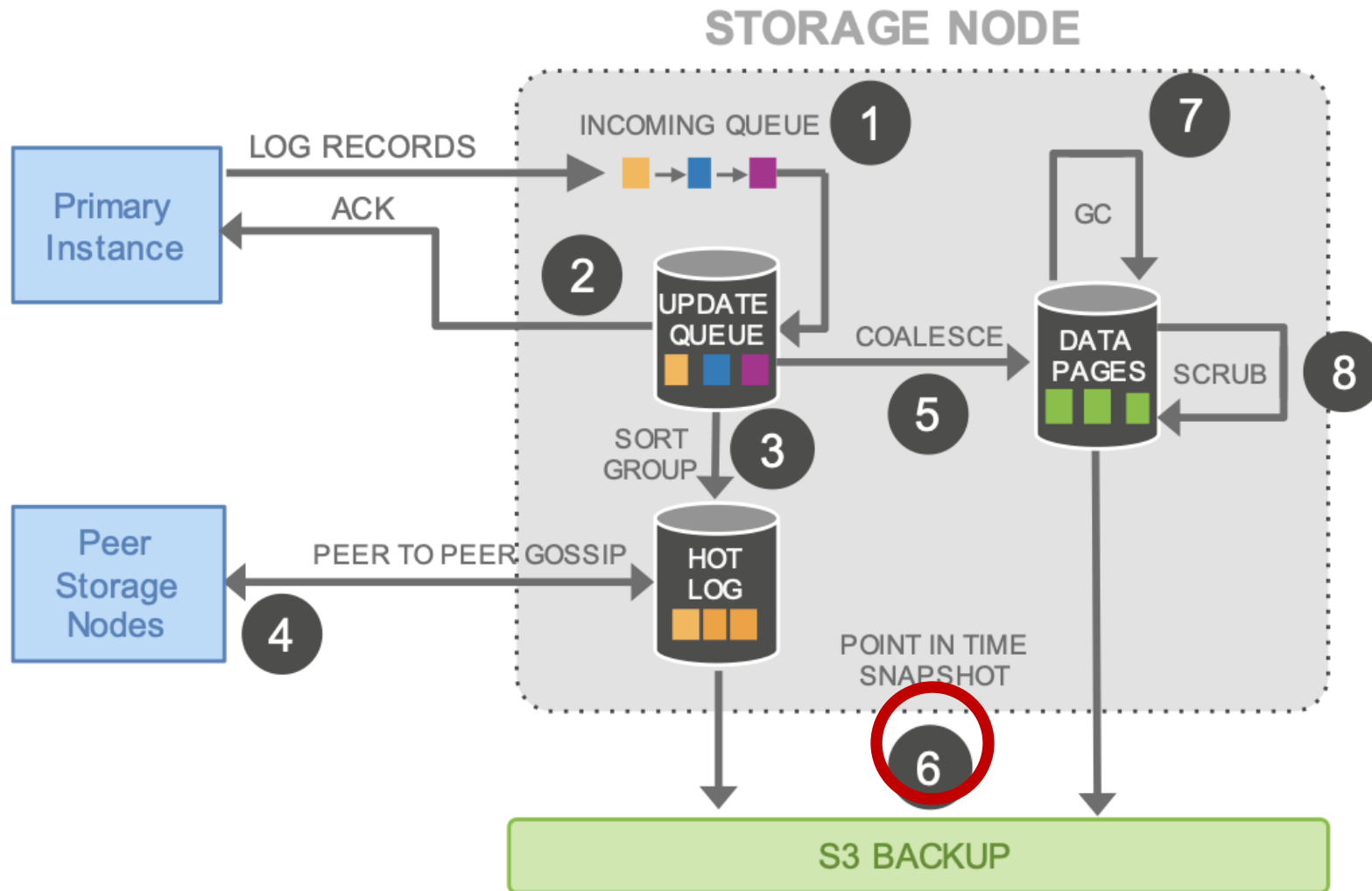


Storage Node



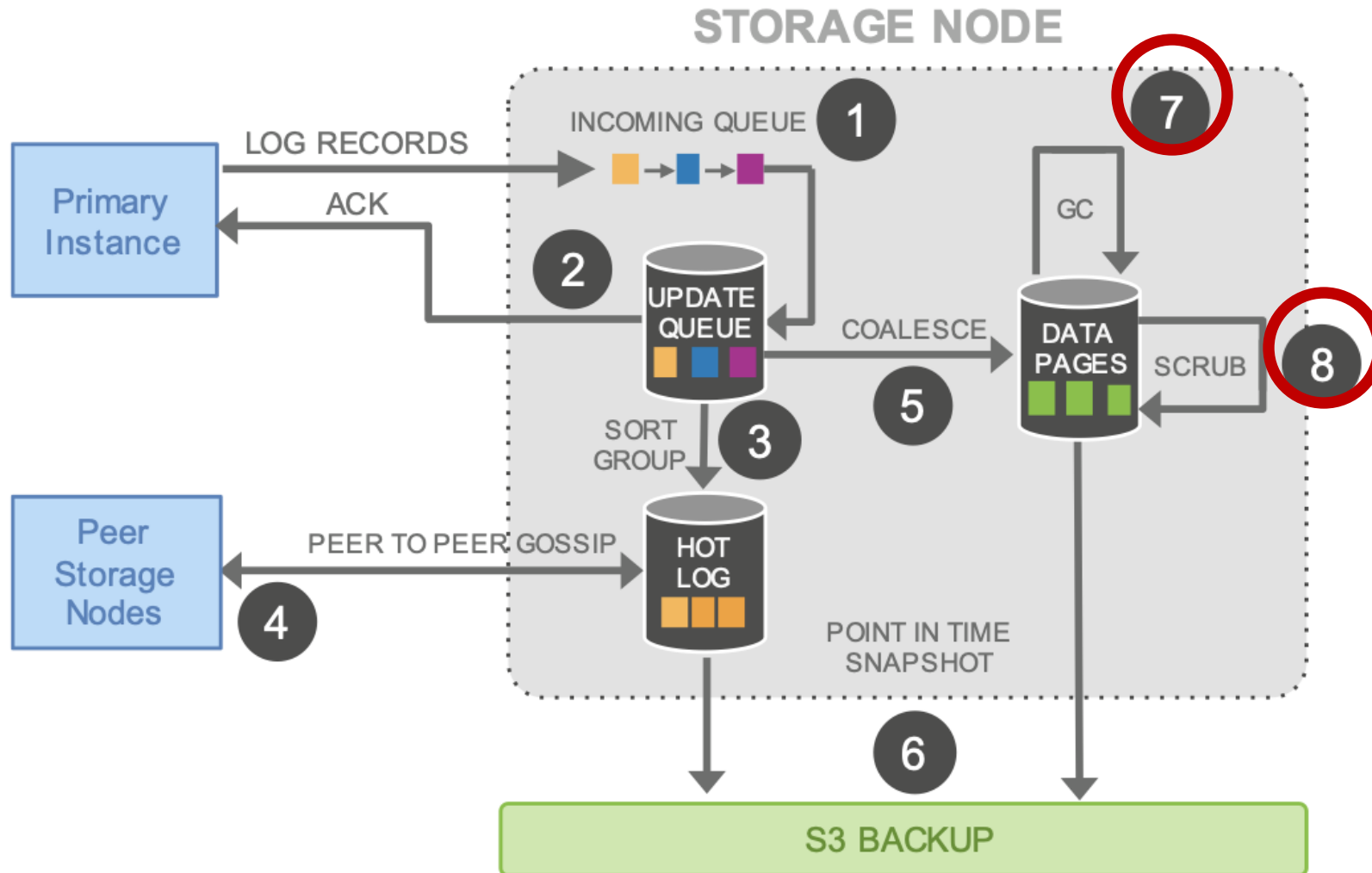
Coalesce log records into data pages

Storage Node



Periodically stage log and pages to S3

Storage Node



Periodically garbage collect old versions and periodically validate CRC code on pages

* Cyclic redundancy check (CRC) is an error-detecting code

Recovery

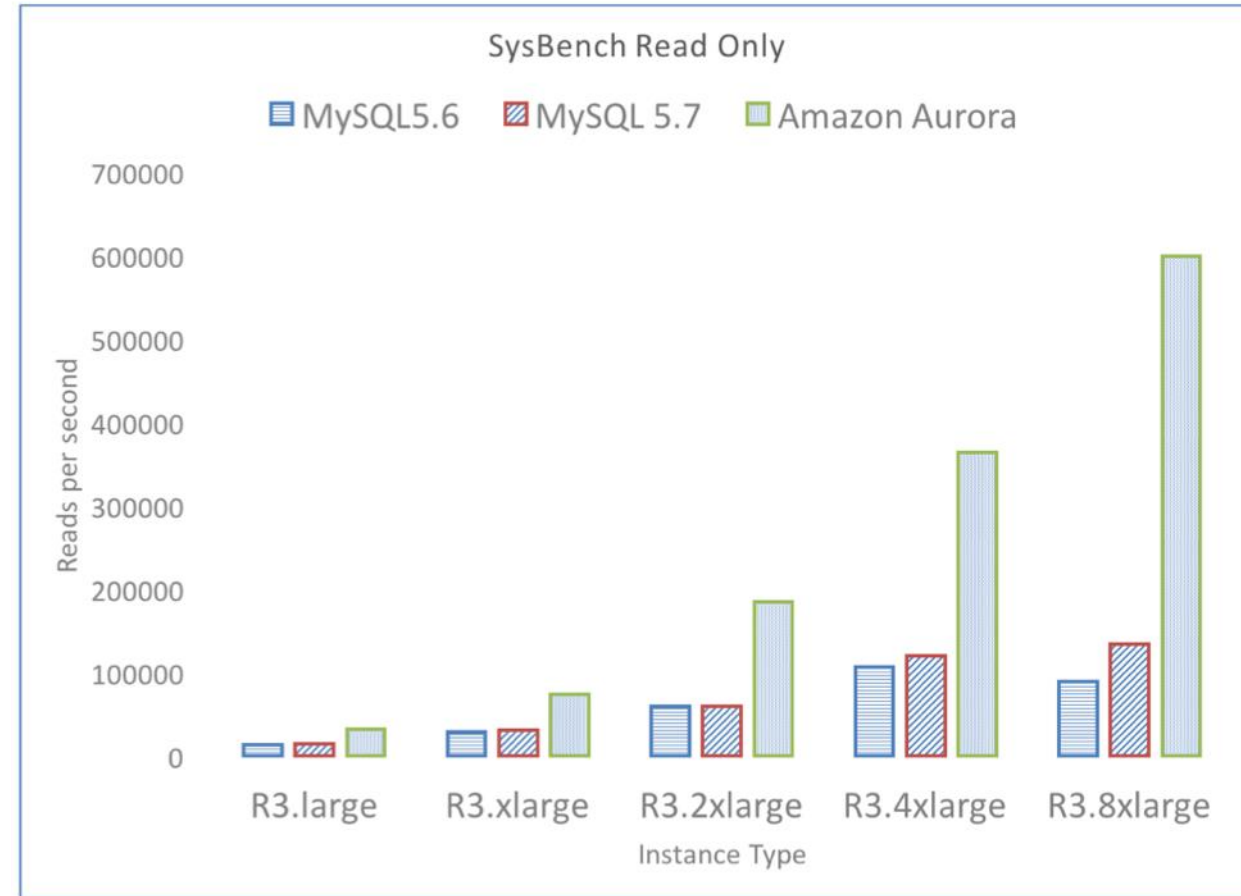
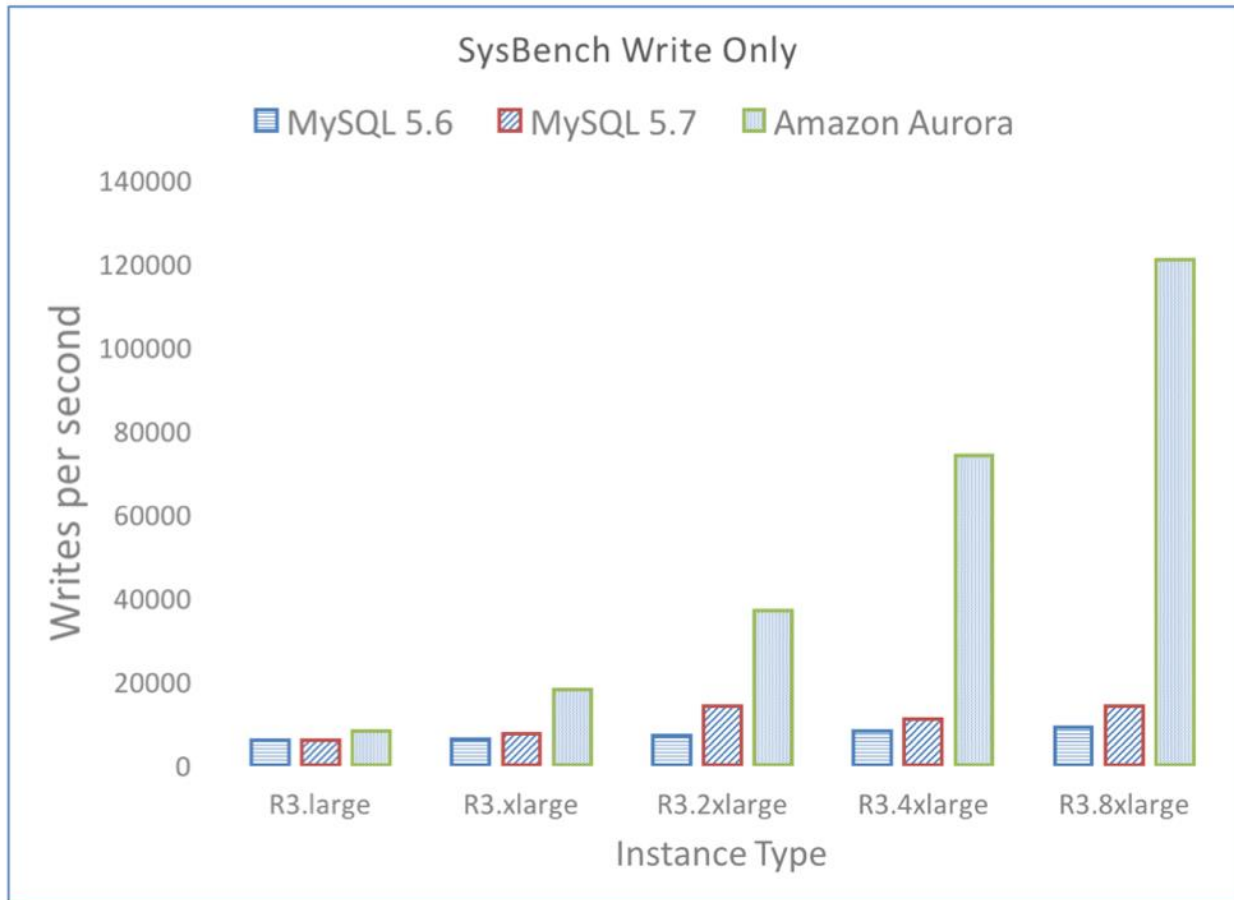
On a high-level:

- Issue quorum read for the log tail, discard everything after
- Database is immediately ready for operations
- Some undo may be necessary, but only done in background
 - Aurora assumes MVCC so uncommitted versions are not visible

Aside: Aurora Multi-Master

- Launched for preview in 2023
- Since deprecated
 - No wide adoption
 - Performance issues due to page-level conflict tracking
- Somewhat replaced by an automatic sharding (i.e., Aurora Limitless)

Evaluation – Aurora vs. MySQL



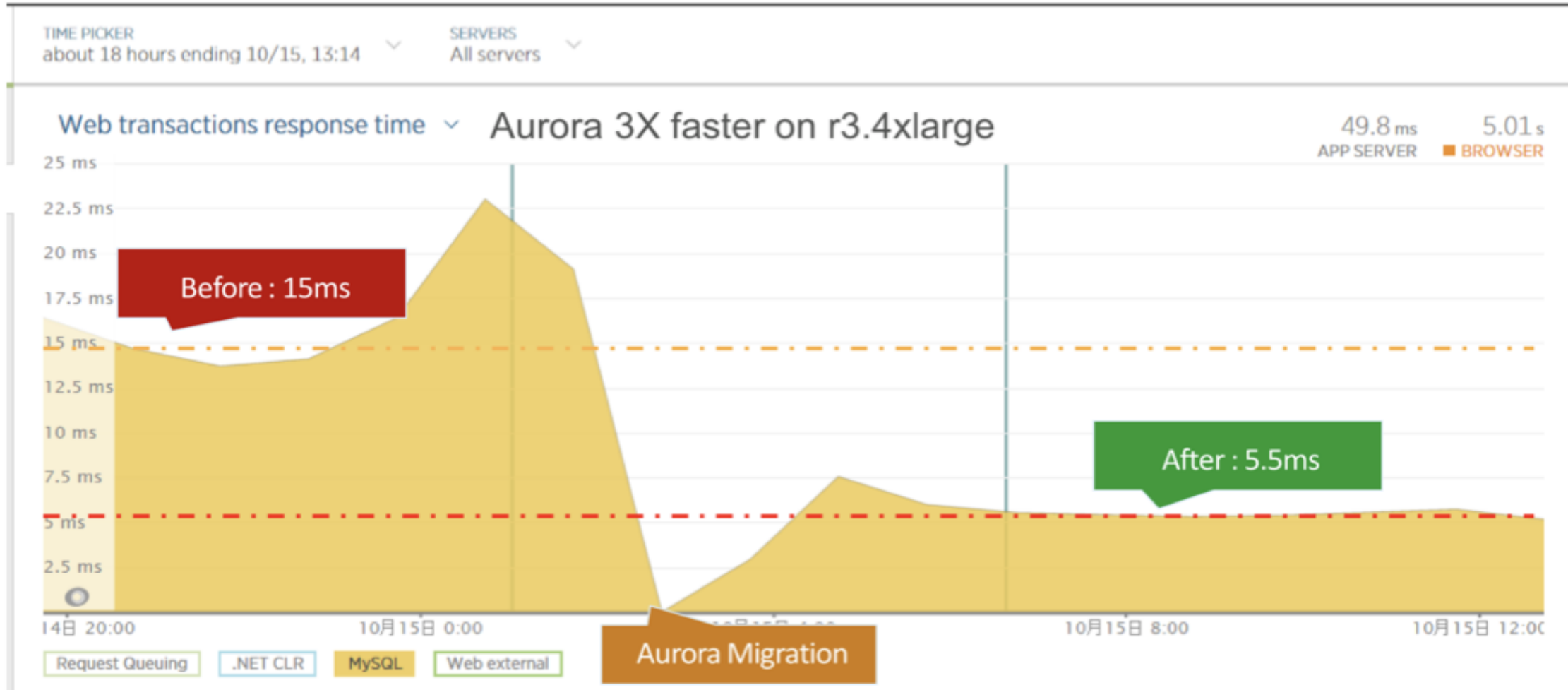
Evaluation – Varying Data Sizes

Table 2: SysBench Write-Only (writes/sec)

DB Size	Amazon Aurora	MySQL
1 GB	107,000	8,400
10 GB	107,000	2,400
100 GB	101,000	1,500
1 TB	41,000	1,200

Performance drops when data does not fit in main memory

Evaluation – Real Customer Workloads



Takeaway: Cloud DB

- New frontier for database engineering
 - Rapidly overtaking legacy systems
- Architectural improvement of classical DB system results in both a cleaner solution and performance/efficiency gains